

Detección de anomalías en servicio de TV-over-IP mediante autoencoder LSTM

Ing. Christopher Charaf

Carrera de Especialización en Inteligencia Artificial

Director: Esp. Ing. María Fabiana Cid (FIUBA)

Jurados:

A designar (pertenencia)
A designar (pertenencia)
A designar (pertenencia)

Ciudad Autónoma de Buenos Aires, junio de 2026

Resumen

Se presenta el desarrollo de un sistema de detección de anomalías aplicado a un servicio de televisión por protocolo de Internet apoyado en interfaces de programación de aplicaciones, en el contexto de una plataforma de video en la nube. El sistema toma métricas técnicas periódicas ya disponibles en infraestructura de monitoreo corporativo, construye ventanas de series temporales multivariadas y entrena un modelo de aprendizaje profundo basado en un autoencoder con redes neuronales recurrentes de memoria a largo plazo para reconstruir el comportamiento esperado y señalar desviaciones mediante el error de reconstrucción frente a un umbral acordado con operaciones. Se describe además la integración con herramientas de alertas y paneles de visualización existentes para acotar el tiempo de detección de incidentes y facilitar la inspección del intervalo temporal asociado a cada evento. Si bien el trabajo se pensó en primera instancia para una empresa cliente, se presentan los resultados obtenidos en un escenario de simulación para validar la robustez del sistema.

En la memoria se expone el marco del problema y el estado del arte, los fundamentos del enfoque elegido, el diseño del procesamiento de datos y del modelo, la implementación del flujo de inferencia y la evaluación con datos históricos. El trabajo integra contenidos de la especialización en inteligencia artificial, en particular aprendizaje automático, aprendizaje profundo aplicado a series temporales e integración con sistemas de observabilidad de software.

Agradecimientos

Índice general

Resumen	I
1. Introducción general	1
1.1. Introducción general al tema	1
1.2. Contexto y motivación	2
1.3. Estado del arte	2
1.4. Objetivos del trabajo	3
2. Introducción específica	5
2.1. Estilo y convenciones	5
2.1.1. Uso de mayúscula inicial para los título de secciones	5
2.1.2. Este es el título de una subsección	5
2.1.3. Figuras	6
2.1.4. Tablas	7
2.1.5. Ecuaciones	8
3. Diseño e implementación	9
3.1. Análisis del software	9
4. Ensayos y resultados	11
4.1. Pruebas funcionales del hardware	11
5. Conclusiones	13
5.1. Conclusiones generales	13
5.2. Próximos pasos	13
Bibliografía	15

Índice de figuras

2.1. Ilustración del cuadrado azul que se eligió para el diseño del logo.	6
2.2. Imagen tomada de la página oficial del procesador ¹	7
2.3. ¿Por qué de pronto aparece esta figura?	7
2.4. Tres gráficos simples.	7

Índice de tablas

1.1. Comparación sintética de enfoques para detección de anomalías en métricas operativas (orientación conceptual).	3
2.1. caption corto	8

Dedicatoria opcional.

Capítulo 1

Introducción general

En el presente capítulo se contextualiza la detección de anomalías en servicios de televisión por protocolo de Internet (TV-over-IP) apoyados en interfaces de programación de aplicaciones (API) y se expone por qué conviene un modelo de aprendizaje profundo frente a umbrales fijos en el monitoreo de infraestructura. Se revisa el estado del arte de forma breve y se enuncian los objetivos del trabajo y las métricas de evaluación.

1.1. Introducción general al tema

Los servicios de video bajo demanda y de transmisión lineal sobre redes IP consolidaron en la última década un modelo donde la calidad percibida por el usuario depende tanto de la disponibilidad de los componentes de *backend* como del desempeño de cadena completa: balanceadores, microservicios, bases de datos, colas de mensajes y sistemas de caché, entre otros. La operación de estos sistemas se apoya de manera casi universal en la recolección periódica de métricas técnicas (latencias, tasas de error, uso de CPU o memoria, etc.) que describen el estado del servicio en el tiempo.

Una anomalía en este contexto operativo, puede entenderse como un patrón de comportamiento en esas señales que se aparta de lo observado cuando el sistema funciona de forma nominal, ya sea por degradación gradual, por un incidente puntual o por una combinación de factores que los umbrales convencionales no caracterizan bien. Detectar esas situaciones de manera temprana importa porque reduce el tiempo de detección de incidentes y acota el impacto en la experiencia de visualización y en los acuerdos de nivel de servicio.

El *machine learning* aporta métodos que aprenden, a partir de datos históricos, qué aspecto tiene el comportamiento “normal” sin fijar a priori todas las reglas. En particular, las redes neuronales recurrentes con unidades de memoria a largo plazo (LSTM, por sus siglas en inglés) permiten modelar dependencias temporales en series multivariadas [1]. Los autoencoders basados en LSTM pueden entrenarse para reconstruir ventanas recientes de métricas; cuando la reconstrucción falla de manera sistemática, el error sirve como señal de anomalía [2]. Este tipo de enfoque es comprensible para un lector familiarizado con monitoreo de infraestructura aunque no especializado en aprendizaje profundo: la idea central es que el modelo aprende un “patrón habitual” y marca desviaciones fuertes respecto de ese patrón.

1.2. Contexto y motivación

La motivación del trabajo final surge de una aplicación real en el sector de plataformas de video empresarial: la organización de referencia provee servicios de *TV-over-IP* sobre infraestructura en la nube y requirió modernizar la forma en que se detectan fallos y degradaciones en producción. Las métricas del servicio se concentran en herramientas estándar de la industria, en particular en un sistema de acumulación de series temporales del tipo de *Prometheus*, con muestreo en el orden de decenas de segundos.

Las prácticas tradicionales de alertas sobre umbrales fijos o reglas manuales siguen siendo útiles para síntomas claros, pero muestran debilidades frente a patrones multivariados, estacionalidades (por ejemplo variaciones por horario comercial o fines de semana) y dependencias que no se traducen en un único umbral intuitivo. Además, externalizar el análisis de métricas a terceros puede no ser deseable desde el punto de vista de privacidad y superficie de ataque.

Por ello se planteó un sistema *on-premise* que consume métricas ya disponibles a través de la *API* del monitoreo, construye ventanas temporales multivariadas, aplica ingeniería de características acorde al dominio (por ejemplo codificación cíclica del tiempo y señales de contexto operativo) y alimenta un autoencoder *LSTM* cuyo error de reconstrucción dispara alertas contextualizadas hacia la plataforma de incidentes y enlaces a tableros de visualización ya utilizados por el equipo de operaciones. La inteligencia artificial se adoptó como herramienta que complementa el criterio humano y centra la atención del operador en intervalos en los que el comportamiento deja de ser coherente con el historial aprendido.

1.3. Estado del arte

La detección de anomalías abarca desde métodos estadísticos y basados en distancia hasta técnicas de aprendizaje profundo adaptadas a datos *secuenciales*; existe una vasta literatura de marcos de referencia y taxonomías [3]. En operaciones de software, los datos son en la práctica series multivariadas en las que distintas métricas capturan facetas acopladas del sistema, lo que motiva modelos que traten la ventana temporal completa en lugar de decidir variable por variable de forma independiente, para capturar no linealidades y dependencias largas.

Los métodos clásicos (por ejemplo controles por umbrales, suavizados exponenciales o *isolation forest* sobre vectores de características) conservan ventajas de simplicidad e interpretabilidad, pero pueden exigir mucho ajuste manual cuando el número de series crece o cuando la normalidad cambia lentamente en el tiempo. Los autoencoders y las arquitecturas recurrentes ofrecen una capa adicional de expresividad para capturar no linealidades y dependencias *largas*; la literatura ha mostrado resultados sólidos con codificadores-decodificadores *LSTM* entrenados para minimizar el error de reconstrucción en ventanas de entrenamiento consideradas normales [2]. Paralelamente, los ecosistemas de observabilidad comercial integran funciones de detección asistida o modelos *proprietarios*; la propuesta de este trabajo se distingue por acoplarse al *stack Prometheus-Grafana-Opsgenie* ya adoptada en la organización y por mantener la configuración, entrenamiento e inferencia bajo el control del operador, buscando un equilibrio entre sensibilidad y falsos positivos calibrable con el equipo de operaciones.

La tabla 1.1 resume de manera orientativa familias de enfoques y su relación típica con datos y métricas de **evaluación**; no pretende ser exhaustiva sino situar la línea elegida en el panorama habitual de ingeniería de confiabilidad y aprendizaje automático.

TABLA 1.1. Comparación sintética de enfoques para detección de anomalías en métricas operativas (orientación conceptual).

Enfoque	Tipo de modelo o regla	Datos típicos	Métricas de desempeño habituales
Umbrales fijos y reglas	Reglas declarativas, comparaciones con constantes	Series univariadas o pocas variables, supuestos estables	Tiempo de detección, acuerdo con incidentes conocidos, carga de falsas alertas
Modelos clásicos de anomalía	Por ejemplo aislamiento, kNN, métodos estadísticos multivariados	Vectores de características por instante o ventana corta	Precisión, exhaustividad (<i>recall</i>), F1 si existe etiquetado parcial
Modelos secuenciales profundos	Autoencoder LSTM , otras RNN, a veces combinadas con atención	Ventanas multivariadas de series temporales	Error de reconstrucción, AUC-PR, F1 sobre eventos etiquetados
Productos comerciales de observabilidad	Algoritmos propietarios, a veces combinados con reglas	Telemetría agregada en plataforma	Definidas por el proveedor; foco en integración

1.4. Objetivos del trabajo

El objetivo general del trabajo consistió en diseñar e implementar un sistema inteligente capaz de detectar anomalías en tiempo de operación compatible con el monitoreo existente en un servicio de **TV-over-IP** basado en **API**, mediante un autoencoder **LSTM** entrenado sobre ventanas de métricas obtenidas desde Prometheus, con integración de alertas y enlaces de contexto con Opsgenie y Grafana, con miras a mejorar la capacidad de reacción frente a incidentes sin depender exclusivamente de umbrales estáticos ni de servicios de terceros para el núcleo analítico.

Los objetivos específicos, formulados de manera verificable, fueron los siguientes:

1. Identificar y preparar las métricas críticas del servicio disponibles vía **API** de Prometheus, incluyendo normalización, variables de contexto temporal (por ejemplo codificación cíclica de la hora) y construcción de ventanas deslizantes acordes al periodo de muestreo del entorno.
2. Entrenar y ajustar un modelo autoencoder **LSTM** capaz de reconstruir el comportamiento nominal del sistema y traducir el error de reconstrucción en una señal de anomalía gobernada por un umbral calibrado con el equipo de operaciones.

3. Integrar el flujo de inferencia con mecanismos de alerta y la generación de enlaces a paneles en Grafana que permitan inspeccionar el intervalo temporal asociado a cada anomalía detectada.
4. Evaluar el sistema sobre datos históricos representativos del régimen normal de operación, reportando al menos la tasa de falsos positivos y la capacidad de detección en escenarios acordados con las partes interesadas, de forma que los resultados en esta memoria permitan contrastar el desempeño frente a líneas base definidas en el capítulo de ensayos.

Las métricas cuantitativas que orientaron la evaluación experimental (y que se desarrollan con detalle metodológico en el capítulo correspondiente) incluyen: (i) *precision y recall* (o pares precisión–exhaustividad equivalentes) frente a conjuntos de eventos anómalos identificados o consensuados con operaciones; (ii) tasa de falsos positivos en períodos libres de incidentes conocidos; (iii) comportamiento del error de reconstrucción y estabilidad frente a cambios lentos de carga. Así el Capítulo 1 fija qué se buscó medir; los valores obtenidos se presentan al desarrollar ensayos y resultados.

Capítulo 2

Introducción específica

Todos los capítulos deben comenzar con un breve párrafo introductorio que indique cuál es el contenido que se encontrará al leerlo. La redacción sobre el contenido de la memoria debe hacerse en presente y todo lo referido al proyecto en pasado, siempre de modo impersonal.

2.1. Estilo y convenciones

2.1.1. Uso de mayúscula inicial para los título de secciones

Si en el texto se hace alusión a diferentes partes del trabajo referirse a ellas como capítulo, sección o subsección según corresponda. Por ejemplo: “En el capítulo 1 se explica tal cosa”, o “En la sección 2.1 se presenta lo que sea”, o “En la subsección 2.1.2 se discute otra cosa”.

Cuando se quiere poner una lista tabulada, se hace así:

- Este es el primer elemento de la lista.
- Este es el segundo elemento de la lista.

Notar el uso de las mayúsculas y el punto al final de cada elemento.

Si se desea poner una lista numerada el formato es este:

1. Este es el primer elemento de la lista.
2. Este es el segundo elemento de la lista.

Notar el uso de las mayúsculas y el punto al final de cada elemento.

2.1.2. Este es el título de una subsección

Se recomienda no utilizar **texto en negritas** en ningún párrafo, ni tampoco texto subrayado. En cambio sí se debe utilizar *texto en itálicas* para palabras en un idioma extranjero, al menos la primera vez que aparecen en el texto. En el caso de palabras que estamos inventando se deben utilizar “comillas”, así como también para citas textuales. Por ejemplo, un *digital filter* es una especie de “selector” que permite separar ciertos componentes armónicos en particular.

La escritura debe ser impersonal. Por ejemplo, no utilizar “el diseño del firmware lo hice de acuerdo con tal principio”, sino “el firmware fue diseñado utilizando tal principio”.

El trabajo es algo que al momento de escribir la memoria se supone que ya está concluido, entonces todo lo que se refiera a hacer el trabajo se narra en tiempo pasado, porque es algo que ya ocurrió. Por ejemplo, "se diseñó el firmware empleando la técnica de test driven development".

En cambio, la memoria es algo que está vivo cada vez que el lector la lee. Por eso transcurre siempre en tiempo presente, como por ejemplo:

"En el presente capítulo se da una visión global sobre las distintas pruebas realizadas y los resultados obtenidos. Se explica el modo en que fueron llevados a cabo los test unitarios y las pruebas del sistema".

Se recomienda no utilizar una sección de glosario sino colocar la descripción de las abreviaturas como parte del mismo cuerpo del texto. Por ejemplo, RTOS (*Real Time Operating System*, Sistema Operativo de Tiempo Real) o en caso de considerarlo apropiado mediante notas a pie de página.

Si se desea indicar alguna página web utilizar el siguiente formato de referencias bibliográficas, dónde las referencias se detallan en la sección de bibliografía de la memoria, utilizando el formato establecido por IEEE en [4]. Por ejemplo, "el presente trabajo se basa en la plataforma EDU-CIAA-NXP [5], la cual...".

2.1.3. Figuras

Al insertar figuras en la memoria se deben considerar determinadas pautas. Para empezar, usar siempre tipografía claramente legible. Luego, tener claro que **es incorrecto** escribir por ejemplo esto: "El diseño elegido es un cuadrado, como se ve en la siguiente figura:"



La forma correcta de utilizar una figura es con referencias cruzadas, por ejemplo: "Se eligió utilizar un cuadrado azul para el logo, como puede observarse en la figura 2.1".



FIGURA 2.1. Ilustración del cuadrado azul que se eligió para el diseño del logo.

El texto de las figuras debe estar siempre en español, excepto que se decida reproducir una figura original tomada de alguna referencia. En ese caso la referencia de la cual se tomó la figura debe ser indicada en el epígrafe de la figura e incluida como una nota al pie, como se ilustra en la figura 2.2.

FIGURA 2.2. Imagen tomada de la página oficial del procesador¹.

La figura y el epígrafe deben conformar una unidad cuyo significado principal pueda ser comprendido por el lector sin necesidad de leer el cuerpo central de la memoria. Para eso es necesario que el epígrafe sea todo lo detallado que corresponda y si en la figura se utilizan abreviaturas entonces aclarar su significado en el epígrafe o en la misma figura.



FIGURA 2.3. ¿Por qué de pronto aparece esta figura?

Nunca colocar una figura en el documento antes de hacer la primera referencia a ella, como se ilustra con la figura 2.3, porque sino el lector no comprenderá por qué de pronto aparece la figura en el documento, lo que distraerá su atención.

Otra posibilidad es utilizar el entorno *subfigure* para incluir más de una figura, como se puede ver en la figura 2.4. Notar que se pueden referenciar también las figuras internas individualmente de esta manera: 2.4a, 2.4b y 2.4c.



(A) Un caption.



(B) Otro.



(C) Y otro más.

FIGURA 2.4. Tres gráficos simples.

El código para generar las imágenes se encuentra disponible para su reutilización en el archivo Chapter2.tex.

2.1.4. Tablas

Para las tablas utilizar el mismo formato que para las figuras, sólo que el epígrafe se debe colocar arriba de la tabla, como se ilustra en la tabla 2.1. Observar que sólo algunas filas van con líneas visibles y notar el uso de las negritas para los encabezados. La referencia se logra utilizando el comando `\ref{<label>}` donde label debe estar definida dentro del entorno de la tabla.

¹Imagen tomada de <https://goo.gl/images/i7C70w>

```

\begin{table}[h]
\centering
\caption[caption corto]{caption largo más descriptivo}
\begin{tabular}{l c c}
\toprule
\textbf{Especie} & \textbf{Tamaño} & \textbf{Valor}\\
\midrule
Amphiprion Ocellaris & 10 cm & \$ 6.000 \\
Hepatus Blue Tang & 15 cm & \$ 7.000 \\
Zebrasoma Xanthurus & 12 cm & \$ 6.800 \\
\bottomrule
\hline
\end{tabular}
\label{tab:peces}
\end{table}

```

TABLA 2.1. caption largo más descriptivo.

Especie	Tamaño	Valor
Amphiprion Ocellaris	10 cm	\$ 6.000
Hepatus Blue Tang	15 cm	\$ 7.000
Zebrasoma Xanthurus	12 cm	\$ 6.800

En cada capítulo se debe reiniciar el número de conteo de las figuras y las tablas, por ejemplo, figura 2.1 o tabla 2.1, pero no se debe reiniciar el conteo en cada sección. Por suerte la plantilla se encarga de esto por nosotros.

2.1.5. Ecuaciones

Al insertar ecuaciones en la memoria dentro de un entorno *equation*, éstas se numeran en forma automática y se pueden referir al igual que como se hace con las figuras y tablas, por ejemplo ver la ecuación 2.1.

$$ds^2 = c^2 dt^2 \left(\frac{d\sigma^2}{1 - k\sigma^2} + \sigma^2 \left[d\theta^2 + \sin^2 \theta d\phi^2 \right] \right) \quad (2.1)$$

Para generar la ecuación 2.1 se utilizó el siguiente código:

```

\begin{equation}
\label{eq:metric}
ds^2 = c^2 dt^2 \left( \frac{d\sigma^2}{1-k\sigma^2} + \right.
\sigma^2 \left[ d\theta^2 + \sin^2 \theta d\phi^2 \right] \left. \right)
\end{equation}

```

Capítulo 3

Diseño e implementación

Todos los capítulos deben comenzar con un breve párrafo introductorio que indique cuál es el contenido que se encontrará al leerlo. La redacción sobre el contenido de la memoria debe hacerse en presente y todo lo referido al proyecto en pasado, siempre de modo impersonal.

3.1. Análisis del software

La idea de esta sección es resaltar los problemas encontrados, los criterios utilizados y la justificación de las decisiones que se hayan tomado.

Se puede agregar código o pseudocódigo dentro de un entorno `lstlisting` con el siguiente código:

```
\begin{lstlisting}[caption= "un epígrafe descriptivo"]
  las líneas de código irían aquí...
\end{lstlisting}
```

A modo de ejemplo, se muestra el fragmento de código [3.1](#):

```
1 #define MAX_SENSOR_NUMBER 3
2 #define MAX_ALARM_NUMBER 6
3 #define MAX_ACTUATOR_NUMBER 6
4
5 uint32_t sensorValue[MAX_SENSOR_NUMBER];
6 FunctionalState alarmControl[MAX_ALARM_NUMBER]; //ENABLE or DISABLE
7 state_t alarmState[MAX_ALARM_NUMBER]; //ON or OFF
8 state_t actuatorState[MAX_ACTUATOR_NUMBER]; //ON or OFF
9
10 void vControl() {
11
12     initGlobalVariables();
13
14     period = 500 ms;
15
16     while(1) {
17
18         ticks = xTaskGetTickCount();
19
20         updateSensors();
21
22         updateAlarms();
23
24         controlActuators();
25
26         vTaskDelayUntil(&ticks, period);
27     }
```

28 }

CÓDIGO 3.1. Pseudocódigo del lazo principal de control.

Capítulo 4

Ensayos y resultados

Todos los capítulos deben comenzar con un breve párrafo introductorio que indique cuál es el contenido que se encontrará al leerlo. La redacción sobre el contenido de la memoria debe hacerse en presente y todo lo referido al proyecto en pasado, siempre de modo impersonal.

4.1. Pruebas funcionales del hardware

La idea de esta sección es explicar cómo se hicieron los ensayos, qué resultados se obtuvieron y analizarlos.

Capítulo 5

Conclusiones

Todos los capítulos deben comenzar con un breve párrafo introductorio que indique cuál es el contenido que se encontrará al leerlo. La redacción sobre el contenido de la memoria debe hacerse en presente y todo lo referido al proyecto en pasado, siempre de modo impersonal.

5.1. Conclusiones generales

La idea de esta sección es resaltar cuáles son los principales aportes del trabajo realizado y cómo se podría continuar. Debe ser especialmente breve y concisa. Es buena idea usar un listado para enumerar los logros obtenidos.

En esta sección no se deben incluir ni tablas ni gráficos.

Algunas preguntas que pueden servir para completar este capítulo:

- ¿Cuál es el grado de cumplimiento de los requerimientos?
- ¿Cuán fielmente se pudo seguir la planificación original (cronograma incluido)?
- ¿Se manifestó algunos de los riesgos identificados en la planificación? ¿Fue efectivo el plan de mitigación? ¿Se debió aplicar alguna otra acción no contemplada previamente?
- Si se debieron hacer modificaciones a lo planificado ¿Cuáles fueron las causas y los efectos?
- ¿Qué técnicas resultaron útiles para el desarrollo del proyecto y cuáles no tanto?

5.2. Próximos pasos

Acá se indica cómo se podría continuar el trabajo más adelante.

Bibliografía

- [1] Sepp Hochreiter y Jürgen Schmidhuber. «Long Short-Term Memory». En: *Neural Computation* 9.8 (1997), págs. 1735-1780.
- [2] Pankaj Malhotra et al. «Long Short Term Memory Networks for Anomaly Detection in Time Series». En: *Proceedings of ESANN*. 2015.
- [3] Varun Chandola, Arindam Banerjee y Vipin Kumar. «Anomaly Detection: A Survey». En: *ACM Computing Surveys* 41.3 (2009), 58:1-58:58.
- [4] IEEE. *IEEE Citation Reference*. 1.^a ed. IEEE Publications, 2016. URL: <http://www.ieee.org/documents/ieeecitationref.pdf> (visitado 26-09-2016).
- [5] Proyecto CIAA. *Computadora Industrial Abierta Argentina*. Visitado el 2016-06-25. 2014. URL: <http://proyecto-ciaa.com.ar/devwiki/doku.php?id=start>.